CSEC 101 // CYBERSECURITY LAB SERIES

02

OTW_BANDIT > 02. Walkthrough

Bandit Walkthrough

> OverTheWire Wargame: Αρχεία, Διασωληνώσεις, Δίκτυο, Cron, SUID, Git & Διαφυγή από Κελύφη

Επίπεδο: Αρχάριος ssh/PuTTY find/grep nc/openssl cron setuid git

```
bandit0@bandit:~$ cat readme
```

Εργαστηριακό Υλικό Κυβερνοασφάλειας: Εκπαιδευτική Χρήση

OverTheWire, Bandit Wargame · Βάσει του write-up του A. Cheron

Αναλυτικές Σημειώσεις: Bandit Walkthrough

Το παρόν εγχειρίδιο αποτελεί ελληνική, εμπλουτισμένη απόδοση του write-up του Alexandre Cheron (`axcheron.github.io/writeups/otw/bandit`) για το wargame **Bandit** της κοινότητας **OverTheWire**.

Περιγραφή και σκοπός του Bandit

Το **Bandit** είναι ένα εκπαιδευτικό παιχνίδι 34 επιπέδων, το οποίο εκτελείται σε πραγματικό εξυπηρετητή (server) Linux. Σε κάθε επίπεδο ο συμμετέχων συνδέεται μέσω **SSH** ως διαφορετικός χρήστης (`bandit0` , `bandit1` , ...) και καλείται να εντοπίσει τον **κωδικό πρόσβασης του επόμενου χρήστη**. Κάθε επίπεδο εισάγει μία νέα εντολή ή έννοια του λειτουργικού συστήματος Linux.

Θεματική ενότητα	Επίπεδα και περιεχόμενο
Αρχεία και ονόματα	0–6: `ls` , `cat` , κρυφά αρχεία, ειδικά ονόματα αρχείων, `file` , `find`
Επεξεργασία κειμένου	7–12: `grep` , `sort | uniq` , `strings` , Base64, ROT13, συμπίεση
Δίκτυο και κλειδιά	13–20: κλειδιά SSH, `nc` , `openssl` , σάρωση θυρών, `diff` , SUID
Αυτοματοποίηση	21–24: `cron` , σενάρια bash, εξαντλητική αναζήτηση
Διαφυγή και Git	25–33: διαφυγή μέσω `more` / `vi` , ιστορικό, κλάδοι, ετικέτες, αποστολή αλλαγών, `$0`

Σημείωση για τους κωδικούς πρόσβασης: οι κωδικοί που εμφανίζονται στις εξόδους των εντολών προέρχονται από το 2018. Το OverTheWire τους αλλάζει περιοδικά και, επομένως, **δεν είναι πλέον έγκυροι**. Το εκπαιδευτικό ενδιαφέρον έγκειται στη μέθοδο επίλυσης και όχι στην ίδια τη συμβολοσειρά.

Οδηγίες χρήσης του εγχειριδίου: επιχειρήστε πρώτα να επιλύσετε κάθε επίπεδο αυτόνομα, συμβουλευόμενοι την επίσημη σελίδα (`overthewire.org/wargames/bandit`). Ανατρέξτε στη λύση μόνο εφόσον δεν είναι δυνατή η πρόοδος, και στη συνέχεια εκτελέστε κάθε εντολή στο δικό σας τερματικό.

Στόχοι του Εργαστηρίου

Με την ολοκλήρωση του Bandit, οι φοιτητές αναμένεται να:

1. Συνδέονται σε απομακρυσμένα συστήματα μέσω `ssh` ή PuTTY και μεταφέρουν αρχεία μέσω `scp` , με χρήση κωδικού πρόσβασης ή ιδιωτικού κλειδιού.

2. Εντοπίζουν αρχεία βάσει ονόματος, μεγέθους, ιδιοκτήτη και δικαιωμάτων (`find` , `file` , `ls -la`).
3. Φιλτράρουν και μετασχηματίζουν κείμενο μέσω διασωληνώσεων (`grep` , `sort` , `uniq` , `tr` , `base64`).
4. Επικοινωνούν με δικτυακές υπηρεσίες μέσω `nc` και `openssl s_client` και εκτελούν σάρωση θυρών.
5. Κατανοούν πώς τα εκτελέσιμα **SUID** και οι εργασίες **cron** ενδέχεται να οδηγήσουν σε κλιμάκωση δικαιωμάτων.
6. Αξιοποιούν το ιστορικό του **git** για την ανάκτηση δεδομένων που έχουν αφαιρεθεί.
7. Διαφεύγουν από περιορισμένα κελύφη (`more` → `vi` , `$0`).

Σύνδεση στον Εξυπηρετητή μέσω SSH

Πριν από το πρώτο επίπεδο, είναι απαραίτητο να κατανοηθεί ο τρόπος σύνδεσης στον εξυπηρετητή του Bandit. Η σύνδεση πραγματοποιείται μέσω του πρωτοκόλλου **SSH (Secure Shell)**, το οποίο παρέχει κρυπτογραφημένο κανάλι επικοινωνίας και πρόσβαση σε κέλυφος (shell) ενός απομακρυσμένου συστήματος. Ό,τι πληκτρολογείται στο τοπικό τερματικό εκτελείται στον απομακρυσμένο εξυπηρετητή.

Στοιχεία σύνδεσης

Παράμετρος	Τιμή
Εξυπηρετητής (host)	`bandit.labs.overthewire.org`
Θύρα (port)	`2220` (και όχι η προεπιλεγμένη θύρα 22 του SSH)
Όνομα χρήστη	`bandit0` για το πρώτο επίπεδο, `banditN` για το επίπεδο N
Κωδικός πρόσβασης	`bandit0` για το πρώτο επίπεδο· οι επόμενοι εντοπίζονται κατά την επίλυση

```
ssh   bandit0  @  bandit.labs.overthewire.org   -p 2220
|     |          |                             |
|     |          |                             └─ θύρα (όχι η προεπιλεγμένη 22)
|     |          └─ ο απομακρυσμένος εξυπηρετητής (host)
|     └─ ο χρήστης, δηλαδή το επίπεδο στο οποίο συνδέεστε
└─ το πρόγραμμα-πελάτης SSH
```

A. Σύνδεση από τερματικό Linux ή macOS

Οι διανομές Linux και το macOS διαθέτουν ενσωματωμένο πελάτη OpenSSH. Ανοίξτε ένα τερματικό (σε Ubuntu: `Ctrl` + `Alt` + `T`) και εκτελέστε:

```
$ ssh bandit0@bandit.labs.overthewire.org -p 2220
The authenticity of host '[bandit.labs.overthewire.org]:2220' can't be established.
ED25519 key fingerprint is SHA256:C2ihUBV7ihnV1wUXRb4RrEcLfXC5CXlhmAAM/urerLY.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[bandit.labs.overthewire.org]:2220' to the list of known hosts.
bandit0@bandit.labs.overthewire.org's password: (πληκτρολογήστε bandit0 και πατήστε Enter)

bandit0@bandit:~$
```

Η διαδικασία περιλαμβάνει τα ακόλουθα βήματα:

1. **Επαλήθευση του εξυπηρετητή.** Κατά την πρώτη σύνδεση εμφανίζεται το αποτύπωμα (fingerprint) του κλειδιού του εξυπηρετητή. Απαντήστε `yes` · το κλειδί αποθηκεύεται στο `~/.ssh/known_hosts` και η ερώτηση δεν επαναλαμβάνεται. (Η ακριβής τιμή του αποτυπώματος ενδέχεται να διαφέρει.)
2. **Εισαγωγή κωδικού πρόσβασης.** Κατά την πληκτρολόγηση του κωδικού **δεν εμφανίζονται χαρακτήρες** στην οθόνη. Η συμπεριφορά αυτή είναι αναμενόμενη και αποτελεί μέτρο ασφαλείας.
3. **Επιτυχής σύνδεση.** Η προτροπή `bandit0@bandit:~$` υποδεικνύει ότι βρίσκεστε πλέον στον απομακρυσμένο εξυπηρετητή.
4. **Αποσύνδεση.** Εκτελέστε `exit` ή πατήστε `Ctrl` + `D`.

Σημείωση για Windows 10/11: τα σύγχρονα Windows περιλαμβάνουν τον ίδιο πελάτη OpenSSH. Η εντολή `ssh bandit0@bandit.labs.overthewire.org -p 2220` εκτελείται αυτούσια σε PowerShell ή Command Prompt, χωρίς εγκατάσταση πρόσθετου λογισμικού.

Β. Σύνδεση από Windows με το PuTTY

Το **PuTTY** είναι δωρεάν γραφικός πελάτης SSH για Windows. Προτείνεται η λήψη του αποκλειστικά από την επίσημη τοποθεσία (`www.chiark.greenend.org.uk/~sgtatham/putty/`), καθώς κυκλοφορούν τροποποιημένες εκδόσεις με κακόβουλο λογισμικό. Το πακέτο εγκατάστασης περιλαμβάνει επίσης τα εργαλεία **PuTTYgen** και **PSCP**, τα οποία απαιτούνται σε μεταγενέστερα επίπεδα.



1. Εκκινήστε το PuTTY. Στην κατηγορία **Session**, συμπληρώστε στο πεδίο **Host Name (or IP address)** την τιμή `bandit.labs.overthewire.org` .
2. Στο πεδίο **Port** ορίστε την τιμή `2220` και στο **Connection type** επιλέξτε **SSH**.
3. (Προαιρετικά) Στο πεδίο **Saved Sessions** πληκτρολογήστε ένα όνομα, π.χ. `bandit` , και πατήστε **Save**, ώστε οι ρυθμίσεις να είναι διαθέσιμες σε επόμενες συνδέσεις.
4. Πατήστε **Open**. Κατά την πρώτη σύνδεση εμφανίζεται το παράθυρο **PuTTY Security Alert** με το αποτύπωμα του κλειδιού του εξυπηρετητή· επιλέξτε **Accept**.

5. Στην προτροπή `login as:` πληκτρολογήστε `bandit0` και, στη συνέχεια, τον κωδικό πρόσβασης. Όπως και στο Linux, ο κωδικός δεν εμφανίζεται κατά την πληκτρολόγηση.

```
login as: bandit0
bandit0@bandit.labs.overthewire.org's password:

bandit0@bandit:~$
```

Αντιγραφή και επικόλληση στο PuTTY: η επιλογή κειμένου με το ποντίκι το αντιγράφει αυτόματα, ενώ το **δεξί κλικ** πραγματοποιεί επικόλληση. Οι συνδυασμοί `Ctrl`+`C` / `Ctrl`+`V` δεν λειτουργούν με τον συνήθη τρόπο, καθώς το `Ctrl`+`C` διακόπτει την εκτελούμενη εντολή.

Σύνδεση με ιδιωτικό κλειδί στο PuTTY (επίπεδα 13 και 16)

Το PuTTY δεν χρησιμοποιεί απευθείας κλειδιά μορφής OpenSSH, αλλά απαιτεί αρχεία μορφής `.ppk`. Η μετατροπή πραγματοποιείται με το **PuTTYgen**:

1. Στο PuTTYgen επιλέξτε **Conversions** → **Import key** (ή **Load**) και ανοίξτε το αρχείο του κλειδιού, π.χ. `sshkey.private`.
2. Επιλέξτε **Save private key** και αποθηκεύστε το ως `bandit14.ppk`.
3. Στο PuTTY, μεταβείτε στο **Connection** → **SSH** → **Auth** → **Credentials** και ορίστε το αρχείο `.ppk` στο πεδίο **Private key file for authentication**.
4. Επιστρέψτε στο **Session** και συνδεθείτε με τον αντίστοιχο χρήστη (π.χ. `bandit14`).

Μεταφορά αρχείων από Windows: το αντίστοιχο της εντολής `scp` είναι το `pscp`: `pscp -P 2220 bandit13@bandit.labs.overthewire.org:sshkey.private .`. Εναλλακτικά, μπορεί να χρησιμοποιηθεί το γραφικό πρόγραμμα **WinSCP** με τα ίδια στοιχεία σύνδεσης.

Συνήθη σφάλματα σύνδεσης

Μήνυμα	Αιτία και αντιμετώπιση
`Connection timed out` / **`Connection refused`**	Λανθασμένη θύρα (συνήθως παράλειψη του `-p 2220`) ή αποκλεισμός της θύρας από το τείχος προστασίας του δικτύου.
`Permission denied, please try again.`	Λανθασμένος κωδικός πρόσβασης ή όνομα χρήστη. Οι κωδικοί διακρίνουν πεζά από κεφαλαία.
`UNPROTECTED PRIVATE KEY FILE!`	Το αρχείο του ιδιωτικού κλειδιού είναι αναγνώσιμο από άλλους χρήστες. Εκτελέστε `chmod 400 <αρχείο>`.
`REMOTE HOST IDENTIFICATION HAS CHANGED!`	Το κλειδί του εξυπηρετητή διαφέρει από το αποθηκευμένο. Εφόσον η αλλαγή είναι γνωστή και αναμενόμενη: `ssh-keygen -R "[bandit.labs.overthewire.org]:2220"`.

Επίπεδο 0 → 1: Αρχική σύνδεση στον εξυπηρετητή

`ssh` · `ls -la` · `cat`

> **Στόχος.** Συνδεθείτε στον εξυπηρετητή `bandit.labs.overthewire.org`, θύρα `2220`, με όνομα χρήστη `bandit0` και κωδικό `bandit0`. Ο κωδικός του επόμενου επιπέδου βρίσκεται στο αρχείο `readme` του προσωπικού καταλόγου.

```
$ ssh bandit0@bandit.labs.overthewire.org -p 2220

bandit0@bandit:~$ ls -la
total 24
drwxr-xr-x  2 root    root    4096 Oct 16 14:00 .
drwxr-xr-x 41 root    root    4096 Oct 16 14:00 ..
-rw-r--r--  1 root    root     220 May 15  2017 .bash_logout
-rw-r--r--  1 root    root    3526 May 15  2017 .bashrc
-rw-r--r--  1 root    root     675 May 15  2017 .profile
-rw-r-----  1 bandit1 bandit0   33 Oct 16 14:00 readme
bandit0@bandit:~$ cat readme
boJ9jbbUNNfktd78OOpsqOltutMc3MY1
```

Επεξήγηση 0: ερμηνεία μιας γραμμής της ls -l

Αρκεί η ανάγνωση του αρχείου `readme` με την εντολή `cat`. Είναι, ωστόσο, σημαντικό να κατανοηθεί **για ποιον λόγο** ο χρήστης έχει δικαίωμα ανάγνωσης:

```
-rw-r-----   bandit1   bandit0   33   readme
|└┘└┘└┘   |         |         |
| |  |  |   |         |         └ μέγεθος σε bytes
| |  |  |   |         └ ομάδα (group)
| |  |  |   └ ιδιοκτήτης (owner)
| |  |  └ λοιποί χρήστες: ---  (κανένα δικαίωμα)
| |  └ ομάδα: r--  (ανάγνωση) ← ο χρήστης bandit0
| └ ιδιοκτήτης: rw- (ανάγνωση/εγγραφή)
└ τύπος: - αρχείο, d κατάλογος, l σύνδεσμος
```

Επίπεδο 1 → 2: Ανάγνωση αρχείου με όνομα «-»

`cat` · `stdin` · `σχετική διαδρομή`

> **Στόχος.** Ο κωδικός βρίσκεται σε αρχείο με όνομα `-` στον προσωπικό κατάλογο.

```
bandit1@bandit:~$ cat ./-
CV1DtqXWVFXTvM2F0k09SHz0YwRINYA9
```

Επεξήγηση 1: η ειδική σημασία του «-»

Για πολλές εντολές το όρισμα `-` σημαίνει **«ανάγνωση από την τυπική είσοδο (stdin)»**. Επομένως, η εντολή `cat -` αναμένει είσοδο από το πληκτρολόγιο και την επαναλαμβάνει. Με τη χρήση **διαδρομής** (`./-`) δηλώνεται ρητά ότι πρόκειται για το αρχείο `-` του τρέχοντος καταλόγου. Η ίδια τεχνική εφαρμόζεται σε κάθε όνομα αρχείου που μπορεί να ερμηνευθεί ως παράμετρος.

Επίπεδο 2 → 3: Ονόματα αρχείων με κενά διαστήματα

`quoting · escaping · αυτόματη συμπλήρωση`

> **Στόχος.** Ο κωδικός βρίσκεται στο αρχείο `spaces in this filename`.

```
bandit2@bandit:~$ ls
spaces in this filename
bandit2@bandit:~$ cat "spaces in this filename"
UmHadQclWmgdLOKQ3YNgjWxGoRMb5luK
```

Επεξήγηση 2: διαχωρισμός ορισμάτων από το κέλυφος

Το κέλυφος διαχωρίζει τη γραμμή εντολών σε ορίσματα **στα κενά διαστήματα**. Χωρίς εισαγωγικά, η `cat` θα αναζητούσε τέσσερα διαφορετικά αρχεία: `spaces`, `in`, `this` και `filename`. Υπάρχουν δύο ισοδύναμες λύσεις: η χρήση εισαγωγικών ή η διαφυγή (escape) κάθε κενού με ανάστροφη κάθετο: `cat spaces\ in\ this\ filename`.

Πρακτική επισήμανση: πληκτρολογήστε `cat sp` και πατήστε `Tab`. Το bash συμπληρώνει αυτόματα το όνομα με τους κατάλληλους χαρακτήρες διαφυγής.

Επίπεδο 3 → 4: Κρυφά αρχεία

`cd · ls -a · dotfiles`

> **Στόχος.** Ο κωδικός βρίσκεται σε **κρυφό** αρχείο εντός του καταλόγου `inhere`.

```
bandit3@bandit:~$ cd inhere/
bandit3@bandit:~/inhere$ ls
bandit3@bandit:~/inhere$ ls -la
total 12
drwxr-xr-x 2 root    root    4096 Dec 28 14:34 .
drwxr-xr-x 3 root    root    4096 Dec 28 14:34 ..
-rw-r----- 1 bandit4 bandit3   33 Dec 28 14:34 .hidden
bandit3@bandit:~/inhere$ cat .hidden
pIwrPrtPN36QITSp3EQaw936yaFoFgAB
```

Επεξήγηση 3: ο ορισμός του κρυφού αρχείου

Στο Linux δεν υφίσταται ιδιότητα «κρυφό αρχείο». Πρόκειται για σύμβαση: κάθε όνομα που αρχίζει με **τελεία** παραλείπεται από την απλή εντολή `ls`. Η παράμετρος `-a` (all) εμφανίζει και αυτά τα αρχεία. Με τον ίδιο τρόπο αποκρύπτονται τα αρχεία ρυθμίσεων (`.bashrc`, `.ssh/`).

Σχετικά με την ασφάλεια: ένα από τα πρώτα βήματα κατά την ανάλυση ενός συστήματος είναι η εκτέλεση της `ls -la` στους καταλόγους ενδιαφέροντος. Κακόβουλα αρχεία συχνά αποκρύπτονται με ονόματα όπως `.cache` ή `...`.

Επίπεδο 4 → 5: Εντοπισμός αρχείου κειμένου βάσει τύπου

`file · χαρακτήρες μπαλαντέρ`

> **Στόχος.** Ο κωδικός βρίσκεται στο μοναδικό αρχείο αναγνώσιμου κειμένου εντός του καταλόγου `inhere`.

```
bandit4@bandit:~/inhere$ file ./-file0*
./-file00: data
./-file01: data
  ...
./-file07: ASCII text
./-file08: data
./-file09: data
bandit4@bandit:~/inhere$ cat ./-file07
koReBOKuIDDepwhWk7jZC0RTdopnAYKh
```

Επεξήγηση 4: αναγνώριση τύπου από το περιεχόμενο

Η εντολή `file` εξετάζει τα πρώτα bytes ενός αρχείου (τα λεγόμενα «magic bytes») και προσδιορίζει τον τύπο του ανεξάρτητα από το όνομα ή την κατάληξη. Με τον χαρακτήρα μπαλαντέρ `*` εξετάζονται όλα τα αρχεία ταυτόχρονα και αναζητείται εκείνο με χαρακτηρισμό **ASCII text**. Το πρόθεμα `./` είναι και εδώ απαραίτητο, καθώς τα ονόματα αρχίζουν με `-`.

Επίπεδο 5 → 6: Αναζήτηση αρχείων βάσει ιδιοτήτων

`find · -size · -readable · ! -executable`

> **Στόχος.** Σε κάποιο σημείο κάτω από τον κατάλογο `inhere` υπάρχει αρχείο που είναι **αναγνώσιμο**, έχει μέγεθος **1033 bytes** και **δεν** είναι εκτελέσιμο.

```
bandit5@bandit:~$ find ./inhere/ -type f -readable ! -executable -size 1033c
./inhere/maybehere07/.file2
bandit5@bandit:~$ cat ./inhere/maybehere07/.file2
DXjZPULLxYr17uwoI01bNLQbtFemEgo7
```

Επεξήγηση 5: κάθε ιδιότητα αντιστοιχεί σε ένα φίλτρο

Παράμετρος	Σημασία
`-type f`	μόνο κανονικά αρχεία (όχι κατάλογοι)
`-readable`	αρχεία αναγνώσιμα από τον τρέχοντα χρήστη
`! -executable`	το `!` αντιστρέφει το φίλτρο: μη εκτελέσιμα αρχεία
`-size 1033c`	ακριβώς 1033 bytes (`c` = bytes, `k` = KiB, `M` = MiB)

Επίπεδο 6 → 7: Αναζήτηση σε ολόκληρο το σύστημα αρχείων

`find / · -user · -group · 2>/dev/null`

> **Στόχος.** Σε κάποιο σημείο του εξυπηρετητή υπάρχει αρχείο με ιδιοκτήτη `bandit7`, ομάδα `bandit6` και μέγεθος **33 bytes**.

```
bandit6@bandit:~$ find / -type f -size 33c -group bandit6 -user bandit7 2>&1 | grep -v "Permission
denied"
/var/lib/dpkg/info/bandit7.password
bandit6@bandit:~$ cat /var/lib/dpkg/info/bandit7.password
HKBPTKQnIay4Fw76bEy8PVxKEDQRKTzs
```

Επεξήγηση 6: τυπική έξοδος και τυπική έξοδος σφαλμάτων

Όταν η αναζήτηση ξεκινά από τη ρίζα `/`, η `find` παράγει εκατοντάδες μηνύματα `Permission denied`. Τα μηνύματα αυτά γράφονται στην τυπική έξοδο σφαλμάτων **stderr** (ροή 2) και όχι στην τυπική έξοδο **stdout** (ροή 1). Η ανακατεύθυνση `2>&1` τα συγχωνεύει με την stdout, ώστε να απορριφθούν από την `grep -v`.

Απλούστερη εναλλακτική: η εντολή `find / ... 2>/dev/null` απορρίπτει απευθείας τη ροή σφαλμάτων, χωρίς χρήση της `grep`.

Επίπεδο 7 → 8: Αναζήτηση συμβολοσειράς σε αρχείο

`grep · find -exec`

> **Στόχος.** Ο κωδικός βρίσκεται στο αρχείο `data.txt`, δίπλα στη λέξη **millionth**.

```
bandit7@bandit:~$ find / -name "data.txt" -exec grep -H 'millionth' {} \; 2>/dev/null
/home/bandit7/data.txt:millionth    cvX2JJa4CFALtqS87jk27qwqGhBM9plV
```

Επεξήγηση 7: η παράμετρος -exec της find

Η παράμετρος `-exec` εκτελεί μια εντολή για κάθε αποτέλεσμα της αναζήτησης· το `{}` αντικαθίσταται από το όνομα του αρχείου και το `\;` σηματοδοτεί το τέλος της εντολής. Δεδομένου ότι το αρχείο βρίσκεται ήδη στον προσωπικό κατάλογο, επαρκεί η εντολή `grep millionth data.txt`.

Επίπεδο 8 → 9: Εντοπισμός μοναδικής γραμμής

`sort · uniq · διασωλήνωση`

> **Στόχος.** Ο κωδικός είναι η **μοναδική γραμμή** του `data.txt` που εμφανίζεται μόνο μία φορά.

```
bandit8@bandit:~$ sort data.txt | uniq -c | grep "1 "
      1 UsvVyFSfZZWbi6wgC7dAFyFuR6jQQUhR
```

Επεξήγηση 8: η αναγκαιότητα της ταξινόμησης

```
data.txt ──> sort ──> uniq -c ──> grep "1 " ──> κωδικός
             |        |           |
             |        |           └─ διατήρηση των γραμμών με μετρητή 1
             |        └─ καταμέτρηση εμφανίσεων κάθε γραμμής
             └─ ομαδοποίηση των ίδιων γραμμών σε διαδοχικές θέσεις
```

Η `uniq` εντοπίζει μόνο **διαδοχικές** όμοιες γραμμές· για τον λόγο αυτό προηγείται η ταξινόμηση με `sort`. Συντομότερη εναλλακτική: `sort data.txt | uniq -u`, η οποία εμφανίζει μόνο τις μοναδικές γραμμές.

Επίπεδο 9 → 10: Εξαγωγή αναγνώσιμων συμβολοσειρών από δυαδικό αρχείο

`strings · grep ^`

> **Στόχος.** Ο κωδικός είναι μία από τις λίγες αναγνώσιμες συμβολοσειρές του `data.txt` και έπεται αρκετών χαρακτήρων `=`.

```
bandit9@bandit:~$ strings data.txt | grep "^=="
========== password
========== isa
========== truKLdjsbJ5g7yyJ2X2R0o3a5HQJFuLk
```

Επεξήγηση 9: η εντολή strings και οι άγκυρες

Η `strings` εξάγει από ένα δυαδικό αρχείο μόνο τις ακολουθίες εκτυπώσιμων χαρακτήρων (εξ ορισμού τουλάχιστον τεσσάρων). Στις κανονικές εκφράσεις της `grep`, ο χαρακτήρας `^` αποτελεί **άγκυρα** για την αρχή της γραμμής, ενώ ο `$` για το τέλος της.

Σχετικά με την ασφάλεια: η `strings` αποτελεί πρώτο βήμα στην ανάλυση άγνωστων εκτελέσιμων και κακόβουλου λογισμικού, καθώς διευθύνσεις URL, διευθύνσεις IP, μηνύματα και κωδικοί είναι συχνά άμεσα ορατοί.

Επίπεδο 10 → 11: Αποκωδικοποίηση Base64

`base64 · κωδικοποίηση και κρυπτογράφηση`

> **Στόχος.** Το αρχείο `data.txt` περιέχει δεδομένα κωδικοποιημένα σε **Base64**.

```
bandit10@bandit:~$ cat data.txt
VGhlIHBhc3N3b3JkIGlzIElGdWt3S0dzRlc4TU9xM0lSRnFyeEUxaHhUTkViVVBSCg==
bandit10@bandit:~$ base64 -d data.txt
The password is IFukwKGsFW8MOq3IRFqrxE1hxTNEbUPR
```

Επεξήγηση 10: η κωδικοποίηση δεν αποτελεί κρυπτογράφηση

Έννοια	Λειτουργία
Κωδικοποίηση (Base64, hex)	Μετατρέπει τη μορφή των δεδομένων για σκοπούς μεταφοράς. Δεν χρησιμοποιεί κλειδί και αντιστρέφεται από οποιονδήποτε.
Κρυπτογράφηση (AES, RSA)	Αποκρύπτει το περιεχόμενο. Χωρίς το κλειδί δεν είναι δυνατή η αντιστροφή.
Συνάρτηση κατακερματισμού (MD5, SHA-256)	Παράγει μονόδρομο αποτύπωμα σταθερού μήκους, το οποίο δεν αντιστρέφεται.

Αναγνώριση του Base64: χρησιμοποιεί αποκλειστικά τους χαρακτήρες `A–Z a–z 0–9 + /`, έχει μήκος πολλαπλάσιο του 4 και συχνά καταλήγει σε `=` ή `==` (συμπλήρωμα, padding).

Επίπεδο 11 → 12: Αποκρυπτογράφηση ROT13

`tr · κρυπτοσύστημα του Καίσαρα`

> **Στόχος.** Στο `data.txt` όλα τα γράμματα έχουν μετατοπιστεί κατά **13 θέσεις**.

```
bandit11@bandit:~$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
The password is 5Te8Y4drgCRfCx8ugdwuEX8KFC6k2EUu
```

Επεξήγηση 11: η tr ως πίνακας αντιστοίχισης

```
πριν:  A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
       | | | | | | | | | | | | | | | | | | | | | | | | | |
μετά:  N O P Q R S T U V W X Y Z A B C D E F G H I J K L M
```

Η `tr` αντικαθιστά κάθε χαρακτήρα του πρώτου συνόλου με τον αντίστοιχο χαρακτήρα του δεύτερου. Επειδή το λατινικό αλφάβητο έχει 26 γράμματα, η διπλή εφαρμογή του ROT13 επαναφέρει το αρχικό κείμενο. Επομένως, η ίδια εντολή εκτελεί τόσο την κωδικοποίηση όσο και την αποκωδικοποίηση.

Επίπεδο 12 → 13: Διαδοχική αποσυμπίεση αρχείου

`xxd` · `gzip` · `bzip2` · `tar` · `/tmp`

> **Στόχος.** Το αρχείο `data.txt` είναι **δεκαεξαδική αποτύπωση (hexdump)** ενός αρχείου που έχει συμπιεστεί επανειλημμένα.

```
# Δημιουργία καταλόγου εργασίας (ο προσωπικός κατάλογος δεν είναι εγγράψιμος)
bandit12@bandit:~$ mkdir /tmp/ax && cp data.txt /tmp/ax && cd /tmp/ax
# Μετατροπή της αποτύπωσης σε δυαδικό αρχείο
bandit12@bandit:/tmp/ax$ xxd -r data.txt data.out
bandit12@bandit:/tmp/ax$ file data.out
data.out: gzip compressed data, was "data2.bin", ...
bandit12@bandit:/tmp/ax$ mv data.out data.gz && gzip -d data.gz
bandit12@bandit:/tmp/ax$ file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/ax$ bzip2 -d data
bandit12@bandit:/tmp/ax$ file data.out
data.out: gzip compressed data, was "data4.bin", ...
bandit12@bandit:/tmp/ax$ mv data.out data.gz && gzip -d data.gz
bandit12@bandit:/tmp/ax$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/ax$ tar -xf data            # → data5.bin (tar)
bandit12@bandit:/tmp/ax$ tar -xf data5.bin       # → data6.bin (bzip2)
bandit12@bandit:/tmp/ax$ bzip2 -d data6.bin      # → data6.bin.out (tar)
bandit12@bandit:/tmp/ax$ tar -xf data6.bin.out   # → data8.bin (gzip)
bandit12@bandit:/tmp/ax$ mv data8.bin data8.gz && gzip -d data8.gz
bandit12@bandit:/tmp/ax$ file data8
data8: ASCII text
bandit12@bandit:/tmp/ax$ cat data8
The password is 8ZjyCRiBWFYkneahHwxCv3wb2a1ORpYL
```

Επεξήγηση 12: μια επαναλαμβανόμενη διαδικασία

Η παράμετρος `-r` (reverse) της `xxd` μετατρέπει μια δεκαεξαδική αποτύπωση στο αρχικό δυαδικό αρχείο. Στη συνέχεια επαναλαμβάνεται η ίδια διαδικασία: **η `file` προσδιορίζει τον τύπο** του αρχείου και εφαρμόζεται το κατάλληλο εργαλείο αποσυμπίεσης.

Αποτέλεσμα της file	Εντολή αποσυμπίεσης
gzip compressed data	`mv x x.gz && gzip -d x.gz` (η gzip απαιτεί την κατάληξη `.gz`)
bzip2 compressed data	`bzip2 -d x` (παράγει `x.out` αν δεν αναγνωρίσει κατάληξη)
POSIX tar archive	`tar -xf x` (εξάγει τα περιεχόμενα αρχεία)
ASCII text	`cat x`: τέλος της διαδικασίας

```
hexdump –xxd -r–> gzip –> bzip2 –> gzip –> tar –> tar –> bzip2 –> tar –> gzip –> ASCII
```

Επίπεδο 13 → 14: Σύνδεση με ιδιωτικό κλειδί

`ssh -i · scp · chmod 400`

> **Στόχος.** Ο κωδικός βρίσκεται στο `/etc/bandit_pass/bandit14` και είναι αναγνώσιμος μόνο από τον χρήστη `bandit14`. Αντί για κωδικό, παρέχεται ένα **ιδιωτικό κλειδί SSH**.

```
bandit13@bandit:~$ ls -l sshkey.private
-rw-r-----  1 bandit14 bandit13 1679 Oct 16 14:00 sshkey.private
bandit13@bandit:~$ exit

# Στον τοπικό υπολογιστή (Linux, macOS ή PowerShell)
$ scp -P 2220 bandit13@bandit.labs.overthewire.org:sshkey.private .
$ chmod 400 sshkey.private
$ ssh -i sshkey.private bandit14@bandit.labs.overthewire.org -p 2220
bandit14@bandit:~$
```

Επεξήγηση 13: αυθεντικοποίηση με ζεύγος κλειδιών

Ο εξυπηρετητής διατηρεί το **δημόσιο** κλειδί του χρήστη bandit14 στο αρχείο `~/.ssh/authorized_keys`. Όποιος κατέχει το αντίστοιχο **ιδιωτικό** κλειδί συνδέεται χωρίς κωδικό πρόσβασης. Το κλειδί μεταφέρεται στον τοπικό υπολογιστή με την `scp` και δηλώνεται με την παράμετρο `-i`. Οι χρήστες του PuTTY ακολουθούν τη διαδικασία μετατροπής σε `.ppk` που περιγράφεται στην ενότητα σύνδεσης.

Συνήθη λάθη: (α) η `scp` απαιτεί **κεφαλαίο** `-P` για τη θύρα, ενώ η `ssh` πεζό `-p`· (β) χωρίς την εντολή `chmod 400`, ο πελάτης SSH εμφανίζει το μήνυμα `UNPROTECTED PRIVATE KEY FILE` και

απορρίπτει το κλειδί.

Επίπεδο 14 → 15: Αποστολή δεδομένων σε θύρα TCP

`nc · TCP · localhost`

> **Στόχος.** Αποστείλετε τον κωδικό του τρέχοντος επιπέδου στη θύρα **30000** του localhost.

```
bandit14@bandit:~$ cat /etc/bandit_pass/bandit14 | nc localhost 30000
Correct!
BfMYroe26WYalil77FoDi9qh59eK5xNr
```

Επεξήγηση 14: το netcat ως δικτυακή διασωλήνωση

Το `nc` (netcat) ανοίγει μια απλή σύνδεση TCP: ό,τι εισέρχεται στην τυπική είσοδό του αποστέλλεται στον εξυπηρετητή, και η απάντηση του εξυπηρετητή εμφανίζεται στην τυπική έξοδο. Λόγω της ευελιξίας του, το netcat θεωρείται βασικό εργαλείο διάγνωσης και δοκιμών δικτύου.

Επίπεδο 15 → 16: Επικοινωνία μέσω SSL/TLS

`openssl s_client · TLS`

> **Στόχος.** Αποστείλετε τον τρέχοντα κωδικό στη θύρα **30001** του localhost, με κρυπτογράφηση SSL/TLS.

```
bandit15@bandit:~$ cat /etc/bandit_pass/bandit15 | openssl s_client -connect localhost:30001 -quiet
depth=0 CN = bandit
verify error:num=18:self signed certificate
verify return:1
Correct!
cluFn7wTiGryunymYOu4RcffSxQluehd
```

Επεξήγηση 15: πελάτης TLS γενικής χρήσης

Αν αποσταλεί απλό κείμενο σε θύρα που αναμένει TLS, ο εξυπηρετητής απαιτεί προηγουμένως **χειραψία (handshake)** και η σύνδεση αποτυγχάνει. Το `openssl s_client` ολοκληρώνει τη χειραψία και στη συνέχεια λειτουργεί όπως το `nc`. Η παράμετρος `-quiet` αποκρύπτει τις λεπτομέρειες του πιστοποιητικού. Η ένδειξη `self signed certificate` είναι αναμενόμενη σε εργαστηριακό περιβάλλον.

Επίπεδο 16 → 17: Σάρωση θυρών

σάρωση θυρών · /dev/tcp · openssl

> **Στόχος.** Αποστείλετε τον τρέχοντα κωδικό σε θύρα του localhost στο εύρος **31000–32000**. Πρέπει να διαπιστωθεί ποιες θύρες είναι ανοιχτές, ποιες υποστηρίζουν SSL και ποια παρέχει τα διαπιστευτήρια. Οι υπόλοιπες επιστρέφουν απλώς ό,τι τους αποσταλεί.

```
bandit16@bandit:~$ for i in {31000..32000}; do
>   (echo > /dev/tcp/localhost/$i) >& /dev/null && echo "Port $i open"
> done
Port 31518 open
Port 31790 open

bandit16@bandit:~$ cat /etc/bandit_pass/bandit16 | openssl s_client -connect localhost:31790 -quiet
Correct!
-----BEGIN RSA PRIVATE KEY-----
MIIEogIBAAKCAQEAvmOkuifmMg6HL2YPIOjon6iWfbp7c3jx34YkYWqUH57SUdyJ
... (25 γραμμές) ...
-----END RSA PRIVATE KEY-----
```

Επεξήγηση 16: σαρωτής θυρών σε μία γραμμή bash

Το bash υποστηρίζει την ειδική διαδρομή `/dev/tcp/HOST/PORT` · η εγγραφή σε αυτήν επιχειρεί σύνδεση TCP. Εάν η σύνδεση επιτύχει, η εντολή επιστρέφει κωδικό επιτυχίας και ο τελεστής `&&` εκτυπώνει τον αριθμό της θύρας.

```
31000 ──────────────────── 31518 ────────── 31790 ──────── 32000
                            |                |
                            echo             SSL ok ──> ιδιωτικό κλειδί RSA
```

Η κατάλληλη θύρα επιστρέφει ιδιωτικό κλειδί. Αποθηκεύστε το τοπικά (π.χ. ως `sshkey`), εφαρμόστε `chmod 400` και χρησιμοποιήστε το για τη σύνδεση ως `bandit17`.

Με εξειδικευμένο εργαλείο: η εντολή `nmap -sV -p 31000-32000 localhost` εντοπίζει τις ανοιχτές θύρες και αναγνωρίζει την υπηρεσία που εκτελείται σε καθεμία.

Επίπεδο 17 → 18: Σύγκριση αρχείων

diff

> **Στόχος.** Ο κωδικός είναι η **μοναδική γραμμή** που διαφέρει μεταξύ των αρχείων `passwords.old` και `passwords.new`.

```
$ ssh -i sshkey bandit17@bandit.labs.overthewire.org -p 2220
bandit17@bandit:~$ diff passwords.old passwords.new
42c42
< 6vcSC74ROI95NqkKaeEC2ABVMDX9TyUr
---
> kfBf3eYk5BPBRzwjqutbbfE887SVc5Yd
```

Επεξήγηση 17: ερμηνεία της εξόδου της diff

Η ένδειξη `42c42` σημαίνει ότι η γραμμή 42 του πρώτου αρχείου **τροποποιήθηκε** (change) στη γραμμή 42 του δεύτερου. Το σύμβολο `<` αφορά το πρώτο (παλαιό) αρχείο και το `>` το δεύτερο (νέο).

Επίπεδο 18 → 19: Παράκαμψη τροποποιημένου .bashrc

`ssh <εντολή> · .bashrc`

> **Στόχος.** Ο κωδικός βρίσκεται στο αρχείο `readme`. Το `.bashrc` έχει τροποποιηθεί ώστε να τερματίζει τη σύνδεση αμέσως μετά την είσοδο.

```
$ ssh bandit18@bandit.labs.overthewire.org -p 2220
Byebye !
Connection to bandit.labs.overthewire.org closed.

$ ssh bandit18@bandit.labs.overthewire.org -p 2220 "cat readme"
IueksS7Ubh8G3DCwVzrTd8rAVOwq3M5x
```

Επεξήγηση 18: διαδραστικό και μη διαδραστικό κέλυφος

Το `.bashrc` φορτώνεται μόνο κατά την εκκίνηση **διαδραστικού** κελύφους. Όταν η εντολή δίνεται ως όρισμα της `ssh`, ο εξυπηρετητής την εκτελεί χωρίς διαδραστικό κέλυφος, οπότε η τροποποίηση δεν ενεργοποιείται. Εναλλακτικά: `ssh -t ... /bin/sh`. (Στο PuTTY, η αντίστοιχη ρύθμιση βρίσκεται στο **Connection → SSH → Remote command**.)

Επίπεδο 19 → 20: Εκτέλεση εντολών μέσω εκτελέσιμου SUID

`setuid · κλιμάκωση δικαιωμάτων`

> **Στόχος.** Χρησιμοποιήστε το εκτελέσιμο **SUID** του προσωπικού καταλόγου για την ανάγνωση του `/etc/bandit_pass/bandit20`.

```
bandit19@bandit:~$ ls -l bandit20-do
-rwsr-x--- 1 bandit20 bandit19 7296 Oct 16 14:00 bandit20-do
bandit19@bandit:~$ ./bandit20-do
Run a command as another user.
  Example: ./bandit20-do id
bandit19@bandit:~$ ./bandit20-do cat /etc/bandit_pass/bandit20
GbKksEFF4yrVs6il55v6gwY5aVje5f0j
```

Επεξήγηση 19: η σημασία του χαρακτήρα s

Ο χαρακτήρας `s` στη θέση του `x` του ιδιοκτήτη (`-rwsr-x---`) υποδηλώνει το bit **SUID**. Ένα τέτοιο αρχείο εκτελείται με τα δικαιώματα του **ιδιοκτήτη** του (bandit20) και όχι του χρήστη που το εκκινεί. Συνεπώς, κάθε εντολή που δίνεται στο `bandit20-do` εκτελείται ως bandit20.

Σχετικά με την ασφάλεια: μετά την απόκτηση πρόσβασης σε ένα σύστημα, η εντολή `find / -perm -4000 2>/dev/null` είναι από τα πρώτα βήματα αναζήτησης κλιμάκωσης δικαιωμάτων. Ένα εσφαλμένα ρυθμισμένο εκτελέσιμο SUID ενδέχεται να παρέχει δικαιώματα **root**.

Επίπεδο 20 → 21: Ακρόαση θύρας με netcat

`nc -l` · ακροατής · setuid

> **Στόχος.** Το εκτελέσιμο SUID `suconnect` συνδέεται σε θύρα του localhost που ορίζεται ως όρισμα, διαβάζει μία γραμμή και τη συγκρίνει με τον κωδικό του bandit20. Σε περίπτωση ταύτισης, αποστέλλει τον κωδικό του bandit21.

```
# Τερματικό 1: ο ακροατής (listener)
bandit20@bandit:~$ nc -lp 31337 < /etc/bandit_pass/bandit20
gE269g2h3mw3pwgrj0Ha9Uoqen1c9DGr

# Τερματικό 2: ο πελάτης
bandit20@bandit:~$ ./suconnect 31337
Read: GbKksEFF4yrVs6il55v6gwY5aVje5f0j
Password matches, sending next password
```

Επεξήγηση 20: αντιστροφή ρόλων πελάτη και εξυπηρετητή



Η παράμετρος `-l` θέτει το netcat σε κατάσταση **ακρόασης** αντί για σύνδεσης, ενώ η ανακατεύθυνση `<` αποστέλλει τον κωδικό στον πρώτο πελάτη που θα συνδεθεί. Για δύο

τερματικά, ανοίξτε δύο ταυτόχρονες συνδέσεις SSH (ή δύο παράθυρα PuTTY).

Με ένα μόνο τερματικό: ο ακροατής εκτελείται στο παρασκήνιο με τον τελεστή `&`: `nc -lp 31337 < /etc/bandit_pass/bandit20 &` και στη συνέχεια `./suconnect 31337`.

Επίπεδο 21 → 22: Εργασίες cron (I): ανάγνωση ρυθμίσεων

`cron · /etc/cron.d`

> **Στόχος.** Ένα πρόγραμμα εκτελείται αυτόματα, σε τακτά χρονικά διαστήματα, από τον χρονοπρογραμματιστή `cron`. Εξετάστε τις ρυθμίσεις στο `/etc/cron.d/` και προσδιορίστε την εντολή που εκτελείται.

```
bandit21@bandit:~$ ls /etc/cron.d/
cronjob_bandit22  cronjob_bandit23  cronjob_bandit24
bandit21@bandit:~$ cat /etc/cron.d/cronjob_bandit22
@reboot bandit22 /usr/bin/cronjob_bandit22.sh &> /dev/null
* * * * * bandit22 /usr/bin/cronjob_bandit22.sh &> /dev/null
bandit21@bandit:~$ cat /usr/bin/cronjob_bandit22.sh
#!/bin/bash
chmod 644 /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv
cat /etc/bandit_pass/bandit22 > /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv
bandit21@bandit:~$ cat /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv
Yk7owGAcWjwMVRwrTesJEwB7WVOiILLI
```

Επεξήγηση 21: ερμηνεία μιας γραμμής crontab

```
*    *    *    *    *    bandit22   /usr/bin/cronjob_bandit22.sh
|    |    |    |    |    |          |
|    |    |    |    |    |          └─ η εντολή που εκτελείται
|    |    |    |    |    └─ ο χρήστης υπό τον οποίο εκτελείται
|    |    |    |    └─ ημέρα εβδομάδας (0–7)
|    |    |    └─ μήνας (1–12)
|    |    └─ ημέρα μήνα (1–31)
|    └─ ώρα (0–23)
└─ λεπτό (0–59)          * = «κάθε»  →  εδώ: κάθε λεπτό
```

Το σενάριο εκτελείται ως **bandit22** και αντιγράφει τον κωδικό του σε αρχείο του `/tmp`, το οποίο είναι αναγνώσιμο από όλους τους χρήστες (`644`). Επομένως, αρκεί η ανάγνωση του αρχείου αυτού.

Επίπεδο 22 → 23: Εργασίες cron (II): ανάλυση σεναρίου

σενάριο bash · md5sum · whoami

> **Στόχος.** Η διάταξη είναι όμοια με εκείνη του προηγούμενου επιπέδου, όμως το σενάριο υπολογίζει δυναμικά το όνομα του αρχείου προορισμού.

```
bandit22@bandit:~$ cat /usr/bin/cronjob_bandit23.sh
#!/bin/bash
myname=$(whoami)
mytarget=$(echo I am user $myname | md5sum | cut -d ' ' -f 1)
echo "Copying passwordfile /etc/bandit_pass/$myname to /tmp/$mytarget"
cat /etc/bandit_pass/$myname > /tmp/$mytarget
bandit22@bandit:~$ echo "I am user bandit23" | md5sum
8ca319486bfbbc3663ea0fbe81326349  -
bandit22@bandit:~$ cat /tmp/8ca319486bfbbc3663ea0fbe81326349
jc1udXuA1tiHqjIsL8yaapX5XIAI6i0n
```

Επεξήγηση 22: ο χρήστης που εκτελεί το σενάριο

Το όνομα του αρχείου είναι το αποτύπωμα MD5 της φράσης «I am user *όνομα χρήστη*». Το σενάριο εκτελείται από το cron ως **bandit23**, επομένως `$myname = bandit23`. Αρκεί ο υπολογισμός του ίδιου αποτυπώματος και η ανάγνωση του αντίστοιχου αρχείου.

Σύνηθες λάθος: η εκτέλεση του σεναρίου ως bandit22. Στην περίπτωση αυτή η `whoami` επιστρέφει `bandit22` και υπολογίζεται εσφαλμένο αποτύπωμα.

Επίπεδο 23 → 24: Εργασίες cron (III): συγγραφή σεναρίου

σενάριο κελύφους · /var/spool · δικαιώματα καταλόγου

> **Στόχος.** Η εργασία cron του bandit24 εκτελεί και στη συνέχεια **διαγράφει** κάθε σενάριο που βρίσκεται στον κατάλογο `/var/spool/bandit24`. Απαιτείται η συγγραφή ενός δικού σας σεναρίου.

```
bandit23@bandit:~$ cat /usr/bin/cronjob_bandit24.sh
#!/bin/bash
myname=$(whoami)
cd /var/spool/$myname
echo "Executing and deleting all scripts in /var/spool/$myname:"
for i in * .*; do
    if [ "$i" != "." -a "$i" != ".." ]; then
        echo "Handling $i"
        timeout -s 9 60 ./$i
        rm -f ./$i
    fi
done
```

```
bandit23@bandit:~$ mkdir /tmp/alex1234 && cd /tmp/alex1234
bandit23@bandit:/tmp/alex1234$ cat > script.sh <<'EOF'
#!/bin/sh
cat /etc/bandit_pass/bandit24 >> /tmp/alex1234/bandit24pass
EOF
bandit23@bandit:/tmp/alex1234$ chmod 777 script.sh /tmp/alex1234
bandit23@bandit:/tmp/alex1234$ cp script.sh /var/spool/bandit24/
# Αναμονή έως ένα λεπτό
bandit23@bandit:/tmp/alex1234$ cat bandit24pass
UoMYTrfrBFHyQXmg6gzctqAwOmw1IohZ
```

Επεξήγηση 23: εκτέλεση κώδικα από χρήστη με υψηλότερα δικαιώματα



Η αναγκαιότητα του `chmod 777` στον κατάλογο: το σενάριο εκτελείται ως **bandit24** και όχι ως ο τρέχων χρήστης. Εάν ο bandit24 δεν έχει δικαίωμα εγγραφής στον `/tmp/alex1234`, το αρχείο εξόδου δεν δημιουργείται. Επιπλέον, το ίδιο το σενάριο πρέπει να είναι εκτελέσιμο.

Σχετικά με την ασφάλεια: εγγράψιμοι κατάλογοι ή σενάρια που εκτελούνται από τον `root` μέσω cron αποτελούν συνήθη οδό κλιμάκωσης δικαιωμάτων σε πραγματικά συστήματα.

Επίπεδο 24 → 25: Εξαντλητική αναζήτηση τετραψήφιου PIN

`brute force` · `seq` · `nc`

> **Στόχος.** Μια υπηρεσία στη θύρα **30002** επιστρέφει τον κωδικό του bandit25 εφόσον της αποσταλεί ο κωδικός του bandit24 μαζί με ένα μυστικό **τετραψήφιο PIN**. Ο μόνος τρόπος εύρεσης του PIN είναι η δοκιμή και των 10.000 συνδυασμών.

```
bandit24@bandit:~$ PASS=$(cat /etc/bandit_pass/bandit24)
bandit24@bandit:~$ for pin in $(seq -w 0000 9999); do echo "$PASS $pin"; done \
>     | nc localhost 30002 | grep -v Wrong
I am the pincode checker for user bandit25. ...
Correct!
The password of user bandit25 is uNG9O58gUE7snukf3bvZ0rxhtnjzSGzG
```

Επεξήγηση 24: μία σύνδεση, 10.000 δοκιμές

Η εντολή `seq -w 0000 9999` παράγει όλους τους αριθμούς του εύρους με προπορευόμενα μηδενικά (`-w` = ίσο πλάτος). Δημιουργούνται όλες οι γραμμές της μορφής «κωδικός PIN» και αποστέλλονται **ταυτόχρονα** μέσω μίας σύνδεσης, προσέγγιση σημαντικά ταχύτερη από 10.000 ξεχωριστές συνδέσεις. Η `grep -v Wrong` αποκρύπτει τις αποτυχημένες απόπειρες.

Σημείωση: το αρχικό write-up δεν περιλάμβανε λύση για το επίπεδο αυτό· η παραπάνω προσέγγιση προστέθηκε στο παρόν εγχειρίδιο. Εάν η υπηρεσία τερματίσει τη σύνδεση πρόωρα, διαχωρίστε το εύρος (π.χ. `0000-4999` και `5000-9999`).

Σχετικά με την ασφάλεια: 10.000 συνδυασμοί εξαντλούνται σε δευτερόλεπτα. Για τον λόγο αυτό, τα πραγματικά συστήματα εφαρμόζουν περιορισμό ρυθμού (rate limiting) ή κλείδωμα λογαριασμού μετά από περιορισμένο αριθμό αποτυχημένων προσπαθειών.

Επίπεδο 25 → 26: Διαφυγή από περιορισμένο κέλυφος μέσω more

`/etc/passwd · more · vi`

> **Στόχος.** Το κέλυφος του χρήστη `bandit26` δεν είναι το `/bin/bash`. Πρέπει να προσδιοριστεί ποιο είναι, πώς λειτουργεί και πώς είναι δυνατή η διαφυγή από αυτό.

```
bandit25@bandit:~$ grep bandit26 /etc/passwd
bandit26:x:11026:11026:bandit level 26:/home/bandit26:/usr/bin/showtext
bandit25@bandit:~$ cat /usr/bin/showtext
#!/bin/sh
export TERM=linux
more ~/text.txt
exit 0
bandit25@bandit:~$ ssh -i bandit26.sshkey bandit26@localhost

  _                     _ _ _   ___   __
 | |__   __ _ _ __   __| |_| |_|__ \ / /_
 | '_ \ / _` | '_ \ / _` | | __| / / '_ \
 |_.__/ \__,_|_| |_|\__,_|_|\__|___\___/
Connection to localhost closed.

# Σμικρύνετε το παράθυρο του τερματικού (περίπου 6 γραμμές) και συνδεθείτε ξανά:
--More--(83%)
# Πατήστε v για την εκκίνηση του vi. Εντός του vi:
:e /etc/bandit_pass/bandit26
5czgV9L3Xx8JPOyRbXh6lQbmIOWvPT6Z
```

Επεξήγηση 25: κάθε πρόγραμμα που εκκινεί άλλο πρόγραμμα αποτελεί πιθανή οδό διαφυγής

```
ssh bandit26 ──> showtext ──> more ──[ v ]──> vi ──[ :set shell / :! ]──> bash
                              |
                              └─ διακόπτεται στο --More-- μόνο αν το κείμενο δεν χωρά
```

Το κέλυφος του bandit26 εμφανίζει ένα αρχείο μέσω της `more` και τερματίζει. Το κείμενο είναι σύντομο, οπότε σε κανονικό παράθυρο η `more` ολοκληρώνεται αμέσως. Με τη **σμίκρυνση του παραθύρου** η `more` αναγκάζεται να διακόψει στην ένδειξη `--More--`, όπου το πλήκτρο `v` εκκινεί τον επεξεργαστή `vi`. Στο PuTTY, το παράθυρο σμικρύνεται απλώς με αλλαγή του μεγέθους του.

Επίπεδο 26 → 27: Εκτέλεση εντολών κελύφους από τον vi

`vi :set shell · :! · setuid`

> **Στόχος.** Από το περιβάλλον του vi, ως bandit26, αποκτήστε κέλυφος και εντοπίστε τον κωδικό του bandit27.

```
:set shell=/bin/bash
:!ls -la
-rwsr-x---  1 bandit27 bandit26 7296 Oct 16 14:00 bandit27-do
-rw-r-----  1 bandit26 bandit26  258 Oct 16 14:00 text.txt
:!./bandit27-do cat /etc/bandit_pass/bandit27
3ba3118a22e93127a4ed485be72ef5ea
```

Επεξήγηση 26: η αναγκαιότητα της εντολής :set shell

Ο vi εκτελεί τις εντολές `:!` μέσω του κελύφους του χρήστη, το οποίο εδώ είναι το `showtext`. Με την εντολή `:set shell=/bin/bash` ορίζεται το bash ως κέλυφος εκτέλεσης. Στη συνέχεια, όπως στο επίπεδο 19, το εκτελέσιμο SUID `bandit27-do` διαβάζει τον κωδικό. Η εντολή `:shell` ανοίγει πλήρες διαδραστικό κέλυφος.

Σχετικά με την ασφάλεια: τεχνικές διαφυγής μέσω επεξεργαστών κειμένου, προγραμμάτων σελιδοποίησης και άλλων εργαλείων καταγράφονται συστηματικά στο **GTFOBins**, βασική πηγή αναφοράς για ελέγχους διείσδυσης.

Επίπεδο 27 → 28: Git (I): κλωνοποίηση αποθετηρίου

`git clone`

> **Στόχος.** Υπάρχει αποθετήριο git στη διεύθυνση `ssh://bandit27-git@localhost/home/bandit27-git/repo`. Ο κωδικός του χρήστη `bandit27-git` ταυτίζεται με αυτόν του `bandit27`.

```
bandit27@bandit:~$ mkdir /tmp/repo123 && cd /tmp/repo123
bandit27@bandit:/tmp/repo123$ git clone ssh://bandit27-git@localhost/home/bandit27-git/repo
Cloning into 'repo'...
Receiving objects: 100% (3/3), done.
bandit27@bandit:/tmp/repo123$ cat repo/README
The password to the next level is: 0ef186ac70e04ea33b4c1853d2526fa2
```

Επεξήγηση 27

Δημιουργήστε κατάλογο εργασίας στο `/tmp` (ο προσωπικός κατάλογος δεν είναι εγγράψιμος), κλωνοποιήστε το αποθετήριο με `git clone` μέσω SSH και διαβάστε το αρχείο README. Τα επόμενα τέσσερα επίπεδα βασίζονται στην ίδια διαδικασία.

Επίπεδο 28 → 29: Git (II): ανάκτηση δεδομένων από το ιστορικό

`git log` · `git checkout`

> **Στόχος.** Όμοια διάταξη, με αποθετήριο στη διεύθυνση `ssh://bandit28-git@localhost/home/bandit28-git/repo`.

```
bandit28@bandit:/tmp/repo1337/repo$ cat README.md
- username: bandit29
- password: xxxxxxxxxx
bandit28@bandit:/tmp/repo1337/repo$ git log --oneline
073c27c fix info leak
186a103 add missing data
b67405d initial commit of README.md
bandit28@bandit:/tmp/repo1337/repo$ git checkout 186a103
HEAD is now at 186a103... add missing data
bandit28@bandit:/tmp/repo1337/repo$ cat README.md
- username: bandit29
- password: bbc96594b4e001778eee9975372716b2
```

Επεξήγηση 28: το ιστορικό του git διατηρεί κάθε αλλαγή

```
b67405d ──────────> 186a103 ──────────────> 073c27c  (HEAD)
initial commit     add missing data       fix info leak
                   ▲ περιέχει τον κωδικό  ▲ τον αφαίρεσε από το README
```

Μια υποβολή (commit) με τίτλο «fix info leak» αποτελεί σαφή ένδειξη. Με την εντολή `git checkout` στην προηγούμενη υποβολή, το αρχείο εμφανίζεται στην κατάσταση πριν από τη «διόρθωση». Εναλλακτικά, οι εντολές `git log -p` ή `git show 073c27c` εμφανίζουν απευθείας τις διαφορές.

Σχετικά με την ασφάλεια: κωδικοί και κλειδιά API που καταχωρίστηκαν σε υποβολή παραμένουν στο ιστορικό ακόμη κι αν αφαιρεθούν μεταγενέστερα. Εργαλεία όπως τα `trufflehog` και `gitleaks` εντοπίζουν ακριβώς τέτοιες διαρροές.

Επίπεδο 29 → 30: Git (III): κλάδοι

`git branch -r` · `git checkout`

> **Στόχος.** Αποθετήριο στη διεύθυνση `ssh://bandit29-git@localhost/home/bandit29-git/repo`.

```
bandit29@bandit:/tmp/plop123/repo$ cat README.md
- username: bandit30
- password: <no passwords in production!>
bandit29@bandit:/tmp/plop123/repo$ git branch -r
  origin/HEAD -> origin/master
  origin/dev
  origin/master
  origin/sploits-dev
bandit29@bandit:/tmp/plop123/repo$ git checkout dev
bandit29@bandit:/tmp/plop123/repo$ cat README.md
- password: 5b90576bedb2cc04c86a9e924ce42faf
```

Επεξήγηση 29

Η φράση «no passwords in production» αποτελεί ένδειξη: ο κλάδος `master` αντιστοιχεί στο περιβάλλον παραγωγής, οπότε η αναζήτηση πρέπει να επεκταθεί σε άλλους κλάδους. Η εντολή `git branch -r` εμφανίζει τους απομακρυσμένους κλάδους, και ο κλάδος `dev` περιέχει τον κωδικό.

Επίπεδο 30 → 31: Git (IV): ετικέτες

`git tag` · `git show`

> **Στόχος.** Αποθετήριο στη διεύθυνση `ssh://bandit30-git@localhost/home/bandit30-git/repo`.

```
bandit30@bandit:/tmp/plop1234/repo$ cat README.md
just an epmty file... muahaha
bandit30@bandit:/tmp/plop1234/repo$ git tag
secret
bandit30@bandit:/tmp/plop1234/repo$ git show secret
47e603bb428404d265f59c42920d81e5
```

Επεξήγηση 30

Ούτε το ιστορικό ούτε οι κλάδοι περιέχουν τον κωδικό. Το git, ωστόσο, υποστηρίζει και **ετικέτες (tags)**, δηλαδή ονομαστικές αναφορές σε αντικείμενα του αποθετηρίου (συνήθως υποβολές, εδώ ένα αντικείμενο blob). Η εντολή `git show` εμφανίζει το περιεχόμενο του αντικειμένου.

Επίπεδο 31 → 32: Git (V): αποστολή αλλαγών

`git add -f` · `.gitignore` · `git push` · `hooks`

> **Στόχος.** Αποθετήριο στη διεύθυνση `ssh://bandit31-git@localhost/home/bandit31-git/repo`. Το README ζητά την αποστολή (push) του αρχείου `key.txt`, με περιεχόμενο `May I come in?`, στον κλάδο `master`.

```
bandit31@bandit:/tmp/plop12345/repo$ echo "May I come in?" > key.txt
bandit31@bandit:/tmp/plop12345/repo$ git add -f key.txt
bandit31@bandit:/tmp/plop12345/repo$ git commit -m "key.txt"
bandit31@bandit:/tmp/plop12345/repo$ git push origin master
remote: Well done! Here is the password for the next level:
remote: 56a9bf19c63d650ce78e6ec0354ee45e
 ! [remote rejected] master -> master (pre-receive hook declined)
```

Επεξήγηση 31

Η αναγκαιότητα του `add -f`: το αποθετήριο περιέχει αρχείο `.gitignore` με τον κανόνα `*.txt`, οπότε το git αγνοεί το αρχείο. Η παράμετρος `-f` επιβάλλει την προσθήκη του. Στον εξυπηρετητή, ένα **pre-receive hook** ελέγχει την αποστολή, εμφανίζει τον κωδικό και, στη συνέχεια, την απορρίπτει. Η απόρριψη είναι αναμενόμενη.

Επίπεδο 32 → 33: Διαφυγή από κέλυφος κεφαλαίων χαρακτήρων

`$0 · διαφυγή από κέλυφος`

> **Στόχος.** Μετά τα επίπεδα του git, ακολουθεί μία ακόμη πρόκληση διαφυγής από περιορισμένο κέλυφος.

```
WELCOME TO THE UPPERCASE SHELL
>> ls
sh: 1: LS: not found
>> $0
$ whoami
bandit33
$ cat /etc/bandit_pass/bandit33
c9c3199ddf4121b10cf581a98d51caee
```

Επεξήγηση 32: μια μεταβλητή χωρίς γράμματα

Το περιορισμένο κέλυφος μετατρέπει κάθε είσοδο σε κεφαλαίους χαρακτήρες πριν από την εκτέλεση, ενώ το Linux διακρίνει πεζά από κεφαλαία (`LS` ≠ `ls`). Η μεταβλητή `$0`, ωστόσο, δεν περιέχει γράμματα προς μετατροπή· πρόκειται για την ειδική μεταβλητή που περιέχει το όνομα του τρέχοντος κελύφους (`sh`), οπότε η εκτέλεσή της εκκινεί κανονικό κέλυφος. Επειδή το πρόγραμμα είναι SUID, το νέο κέλυφος εκτελείται ως **bandit33**.

Εναλλακτική προσέγγιση: το αρχικό write-up εκκινεί από το σημείο αυτό τον `vim` και διαβάζει τον κωδικό με την εντολή `:r /etc/bandit_pass/bandit33`.

Επίπεδο 33: Ολοκλήρωση του παιχνιδιού

> **Στόχος.** Δεν υπάρχουν περαιτέρω επίπεδα. Συνδεθείτε για να επιβεβαιώσετε τον τελευταίο κωδικό.

```
bandit33@bandit:~$ cat README.txt
Congratulations on solving the last level of this game!
At this moment, there are no more levels to play in this game. ...
In the meantime, you could play some of our other wargames.
```

Ως επόμενο βήμα προτείνονται τα wargames **Leviathan** (σύντομο, με έμφαση στα εκτελέσιμα αρχεία) και **Natas** (ασφάλεια διαδικτυακών εφαρμογών).

Παράρτημα Α: Σύνοψη εντολών ανά επίπεδο

Επίπεδο	Κύρια εντολή	Έννοια
0–3	`ssh` , `ls -la` , `cat ./-`	Σύνδεση, ειδικά ονόματα, κρυφά αρχεία
4–6	`file` , `find`	Αναζήτηση βάσει τύπου, μεγέθους, ιδιοκτήτη
7–9	`grep` , `sort | uniq` , `strings`	Φιλτράρισμα κειμένου με διασωληνώσεις
10–12	`base64 -d` , `tr` , `xxd -r`	Κωδικοποιήσεις και συμπίεση
13	`scp` , `ssh -i`	Αυθεντικοποίηση με κλειδί
14–16	`nc` , `openssl s_client` , `/dev/tcp`	TCP, TLS, σάρωση θυρών
17–18	`diff` , `ssh host "cmd"`	Σύγκριση αρχείων, μη διαδραστικό κέλυφος
19–20	`./bandit20-do` , `nc -l`	SUID, ακρόαση θύρας
21–23	`cat /etc/cron.d/*` , `md5sum`	Ανάλυση και εκμετάλλευση εργασιών cron
24	`seq -w` + `nc`	Εξαντλητική αναζήτηση
25–26	`more` → `vi` → `:!`	Διαφυγή από περιορισμένο κέλυφος
27–31	`git log / branch / tag / push`	Ευαίσθητα δεδομένα σε αποθετήρια git
32	`$0`	Παράκαμψη φίλτρου εισόδου

Παράρτημα Β: Γλωσσάρι

Όρος	Σημασία
SSH	Πρωτόκολλο κρυπτογραφημένης απομακρυσμένης πρόσβασης σε κέλυφος (θύρα 22· εδώ 2220)
PuTTY / PuTTYgen	Γραφικός πελάτης SSH για Windows / εργαλείο διαχείρισης και μετατροπής κλειδιών
stdin / stdout / stderr	Τυπική είσοδος / έξοδος / έξοδος σφαλμάτων (ροές 0 / 1 / 2)
Διασωλήνωση (`|`)	Σύνδεση της stdout μιας εντολής με τη stdin της επόμενης
Dotfile	Αρχείο του οποίου το όνομα αρχίζει με `.` · δεν εμφανίζεται με την απλή `ls`
Base64	Κωδικοποίηση δυαδικών δεδομένων σε κείμενο (δεν αποτελεί κρυπτογράφηση)
ROT13	Μετατόπιση γραμμάτων κατά 13 θέσεις· παραλλαγή του κρυπτοσυστήματος του Καίσαρα
Hexdump	Αναπαράσταση bytes σε δεκαεξαδική μορφή (`xxd`)
TLS / SSL	Πρωτόκολλα κρυπτογράφησης μιας σύνδεσης TCP
Σάρωση θυρών	Έλεγχος των θυρών ενός συστήματος που δέχονται συνδέσεις
SUID	Εκτέλεση αρχείου με τα δικαιώματα του ιδιοκτήτη του
Cron	Χρονοπρογραμματιστής περιοδικών εργασιών
Εξαντλητική αναζήτηση	Δοκιμή όλων των πιθανών τιμών (brute force)
Διαφυγή από κέλυφος	Απόκτηση πλήρους κελύφους μέσα από περιορισμένο πρόγραμμα (shell escape)
Git hook	Σενάριο που εκτελείται αυτόματα σε συμβάντα του git (π.χ. αποστολή αλλαγών)

Παράρτημα Γ: Λίστα ελέγχου δεξιοτήτων

- [ ] Συνδέομαι με `ssh` ή PuTTY σε μη προεπιλεγμένη θύρα, με κωδικό ή ιδιωτικό κλειδί
- [ ] Μετατρέπω κλειδί OpenSSH σε μορφή `.ppk` με το PuTTYgen
- [ ] Διαχειρίζομαι αρχεία με ιδιαίτερα ονόματα (`-` , κενά, αρχική τελεία)

- [ ] Συντάσσω εντολή `find` με φίλτρα μεγέθους, ιδιοκτήτη και δικαιωμάτων
- [ ] Εξηγώ τη διαφορά μεταξύ `2>/dev/null` και `2>&1`
- [ ] Συνθέτω διασωληνώσεις με `sort`, `uniq`, `grep`, `tr`
- [ ] Διακρίνω την κωδικοποίηση, την κρυπτογράφηση και τον κατακερματισμό
- [ ] Επικοινωνώ με δικτυακή υπηρεσία μέσω `nc` και `openssl s_client`
- [ ] Εξηγώ τη σημασία του `s` στο `-rwsr-x---` και τους κινδύνους που συνεπάγεται
- [ ] Ερμηνεύω μια γραμμή crontab και προσδιορίζω τον χρήστη υπό τον οποίο εκτελείται
- [ ] Αναζητώ ευαίσθητα δεδομένα στο ιστορικό, τους κλάδους και τις ετικέτες ενός αποθετηρίου git
- [ ] Διαφεύγω από περιορισμένο κέλυφος μέσω `vi` ή `$0`

Πηγή: A. Cheron, «[OTW] Write-up for the Bandit Wargame», axcheron.github.io. Ελληνική απόδοση και επεξηγήσεις για το μάθημα CSEC 101.